

Efficient Deployment of a Web Application in Serverless Environment



Vivek Sharma, Akhilesh Kumar Singh, and Manish Raj

Abstract In the recent era to gain speed up, scale-up/down, and for cost-effective features, people are moving toward services provided by AWS or other computing paradigms, (salesforce, etc.) i.e., AWS Lambda which is applicable to build a serverless environment. Serverless environment doesn't mean there is no server but tells AWS to not only give all services which are provided by the server but also give metering on a pay per second basis. This project will entertain users with a full stack-based interface where they can request a unicorn ride from any place they want.

Keywords Amazon Web Service Lambda · Amazon Application Programming Interface · Gateway · Amazon S3 · Amazon DynamoDB · Amazon Cognito and serverless architecture

1 Introduction

Serverless in Amazon Web Service simply means serverless user interface. Serverless user interface is that which doesn't require to supervise servers. By working in a serverless environment, we are free from these inefficiencies, i.e., compartmentalization of resources, scalability, and standardization. To be a serverless platform, it is necessary for it to provide several functionalities. There is no server supervisioning, adjustable scalability, and no idle capacity.

The AWS cloud consists of many services which directly or indirectly come in the scenario of a serverless web application. These are Amazon Web Service Lambda for

V. Sharma (✉) · A. K. Singh · M. Raj
GLAU Mathura (UP), Mathura, India
e-mail: viveksharma.cea@gla.ac.in

A. K. Singh
e-mail: akhileshkr.singh@gla.ac.in

M. Raj
e-mail: manish.raj@gla.ac.in

computation [1], Amazon Application Programming Interface gateway for APIs [2], Amazon Simple Warehouse Service (Amazon S3) to provide storage [3], Amazon DynamoDB for databases [4], Simple Notification and Queue Service by Amazon (ASNQS, a combination of SNS and SQS service by Amazon) [5] and for inter-process managing [6], Amazon Web Server Step Functions [7] and Amazon Cloud Watch Events [8] for orchestration.

2 Literature Review

The first public cloud infrastructure vendor to offer abstract serverless computing AWS Lambda in 2014 was Amazon. Back in the year 2006, a cloud warehouse and storage service were launched by Amazon: Amazon Web Service S3 which presents a warehouse service that performs functions without the requirement of handling the maintenance of the servers. Later in late 2014, a serverless platform was launched by Amazon Web Services in which computation service became serverless. And it was from this point onwards that the new paradigm of cloud revolution began not so long after, that in 2016, other major cloud vendors released serverless platforms to compete with already existing Amazon Web Services, such as Google, Microsoft and IBM. Almost instantly, serverless computing started gaining popularity with organizations opting for the new technology to transform their businesses.

3 Preliminaries

The main emphasis of the paper presents an approach to provide high availability, flexible scaling, and reducing the overhead involved in server management. To achieve this, certain AWS services are used that are the key components in obtaining a serverless environment. Basically, the 4-step scheme is followed for developing a serverless environment. The four steps are—Hosting a non-dynamic web HTML, Management of end users, Building a serverless back end, and Restful Application Programming Interface.

4 Proposed Serverless Web Application

4.1 Description

In the proposed scenario for serverless web services-based application, we have developed a simple user interface which enables users to have a unicorn ride through the serverless platform from anywhere. In this, the user will request a unicorn from

the front-end and their request will be accepted through restful web services (back end) and the unicorn will be dispatched to the nearby place. This project has been done on different levels. These levels are

1. **Computation level:**

The computation level manages the queries coming via external systems or end users. Authorization of the requests is also done here. It has the runtime platform where you can deploy your business logic.

- Amazon Web Services Lambda [1] runs serverless applications on managed platform which enables microservices, deployment, etc.
- Using Amazon Web Services Application Programming Interface gateway [2], we can deploy our business logic on fully managed rest Application Programming Interface integrated with Amazon Web Services Lambda.

The data level gives secure storage for states which our business logic has.

2. **Data level:**

- Amazon DynamoDB [4] provides us with managed no Structured Query Language database for storage purposes.
- Amazon Simple Storage Service [3] provides object storage for any range of use cases.

2. **User Management and Identity Level:**

User Management and Identity Level provides authentication, authorization, and identity to the customers (internal or external both).

- Amazon Cognito [9] provides simple, secured end-user registration, sign up, log in and log out, and user control.

4.2 Working of the Proposed Serverless Web Application

In our proposed scenario, Amazon Simple Storage Service helps in hosting the non-dynamic resources feasible for the web application. The simple logic behind this architecture is that you have to store all your static resources which include HyperText Markup Language, Cascading Style Sheet, Images, Multimedia, and other files. The user can directly view the content through the URL address spread by Amazon S3. You need not have to provision or supervise any server for it. Firstly, in Amazon Management console, select S3 under storage in service [10]. Create an S3 bucket for storing your static resources using a globally unique name [11], upload your content into it by selecting or dragging files, add the bucket policy to it by using the permission tab which allows the public access, and then save the bucket policy code. Select a static hosting card where the first field can be filled with index.html and other can be

made blank but note that before saving this, keep in mind the endpoint URL address because after this you are going to use your web application using this URL address only, and then save it [12]. In Amazon Cognito you are going to register yourself in the web application which requires your email address and password and then it sends a verification code to your email address for authentication, and authorization. Then at login, a javascript function will communicate with Amazon Cognito and get back a set of keys for JSON Web Token which claims the identity of the user. Go to Cognito under services and select the manage your user pool and then click create user pool; create your pool with Wild Rydes and select review default and then create pool (write down pool id ap-south-1_qMZk3inhv), add an application to the user pool; from general settings click app clients and from there create app client and name this from Wild Rydes web application and uncheck the generate secret option and then save it and write down the application client id 386914j606upaa25ab42b6p5vo. Then update the config.js from pool id, application client id, invoke URL address, etc., and then resubmit it to S3 bucket which was made. Then on the web application register yourself. If registration becomes successful, you should get the notification that the application programming interface has not been configured [13–16].

AWS Lambda and Amazon DynamoDB are for building the serverless back ends for the request handling. You will implement the lambda function, each time user requests a unicorn. The module will choose a unicorn amongst a fleet and save the query in Amazon DynamoDB; once done reply to the front-end app with information of the unicorn going to be dispatched. Go to dynamo under services, select create table and name your table with Rides and the key with Rideid having the type string and by checking default checking box, make your table. (Note the ARN arn:aws:dynamodb:ap-south-1:741534142387: table/Rides). Then, to know what Amazon services can interact with the need to have an IAM role for which IAM under services should be selected, create your role by going to the left navigation bar 'roles'. Select lambda for the role type and then click next, give your role name with Wild Rydes Lambda and then create a role. Then on clicking on the role, select add inline policy, choose the service DynamoDB. Once done select the action put_item, add the ARN of the table in resources. Choose review policy and click creates then goes the lambda menu under service tab and creates a function over there. Write RequestUnicorn in name field and choose node.js 6.10 at execution time, write the existing role created and click to create function and update the index.js code with requestunicorn.js. Once done then click save. Then configure test event and add your event name TestRequestEvent and update the code. When the tested log appears, then this phase is completed [17–20].

Through restful APIs, you are going to invoke your lambda function and this static hosted website will convert into a dynamically hosted one by bringing the source side javascript which allows Asynchronous JavaScript and XML calls the shown Application Programming Interface. In Amazon Management Console, go to Application Programming Interface gateway under services and then click on create Application Programming Interface and then on new Application Programming Interface, name your Application Programming Interface (Wild Rydes) and in endpoint type select edge optimized. Then click on the create Application Programming Interface

button. We are going to configure our Application Programming Interface authorizer which we have created to consume the user pool we made in Amazon Cognito. Under your newly created Application Programming Interface, choose authorizer, choose to create authorizer Application Programming Interface, name your authorizer, select Cognito, and fill the region where you created your user pool in Amazon Cognito. Then, enter your user pool name and then write Authorization in token source and then click create Go to resources under your Application Programming Interface and in action click on create resource, enter your resource name (Ride) and ensure that your resource path is ride, choose Enable Application Programming Interface Gateway CORS and then choose create resource button for the recently created resource, under action click create method. Select post on the new drop-down that appears, select lambda function for integration type and then check on the checkbox of use lambda proxy integration, select the region in which we have created our lambda function, and enter the name of our previously created lambda function and click save.

Choose deploy Application Programming Interface, and select new stage in the development stage drop-down menu. Enter stage name (prod) and choose deploy Application Programming Interface. Note the invoke Application Programming Interface URL. Update your config.js file and upload the changed file to Amazon Storage. Visit ride.html under your website domain; if redirected to the sign-in page, then sign in.

References

1. <https://aws.amazon.com/lambda/>
2. <https://aws.amazon.com/api-gateway/>
3. <https://aws.amazon.com/s3/>
4. <https://aws.amazon.com/dynamodb/>
5. <https://ap-south-1.console.aws.amazon.com/sns/v3/home?region=ap-south-1#/dashboard>
6. <https://ap-south-1.console.aws.amazon.com/sqs/home?region=ap-south-1>
7. <https://ap-south-1.console.aws.amazon.com/states/home?region=ap-south-1#/homepage>
8. <https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#>
9. <https://aws.amazon.com/cognito/>
10. <https://s3.console.aws.amazon.com/s3/buckets/wildrydes-dolly-sen/?region=ap-south-1&tab=properties>
11. <https://s3.console.aws.amazon.com/s3/buckets/wildrydes-dolly-sen/?region=ap-south-1&tab=permissions>
12. <https://s3.console.aws.amazon.com/s3/buckets/wildrydes-dolly-sen/?region=ap-south-1&tab=overview>
13. <https://ap-south-1.console.aws.amazon.com/cognito/home?region=ap-south-1?>
14. Miller M (2008) Cloud computing: web-based applications that change the way you work and collaborate online. Que Publishing, Indianapolis
15. Reese G (2009) Cloud application architectures. O'Reilly Media, Sebastopol, CA
16. Sun (2009a) A Guide to Getting Started with Cloud Computing. Sun white paper. https://www.sun.com/offers/docs/cloud_computing_primer.pdf. Accessed: 10 June 2009
17. Foster I, Zhao Y, Raicu I, Lu S (2008) Cloud computing and grid computing 360-Degree Compared

18. RightScale (2008) Define Cloud Computing. RightScale Blog, 26 May 2008. <http://blog.rightscale.com/2008/05/26/define-cloud-computing/>. Accessed 9 Jun 2009
19. Gartner (2008c) Gartner Says Worldwide SaaS Revenue in the Enterprise Application Markets Will Grow 27 Per Cent in 2008. Gartner press release, 22 October 2008
20. Singh AK, Singh P An approach for web based GIS Route Finder System, Int J Advanc Res Comput Sci Softw Eng (IJARCSSE)